Image Basic Transforms

Image Basic Transforms

- Preface
- Example Results
- Principles
  - Why Can Images Be Processed by OpenCV
  - Four Processing Modes
- Code Overview
  - Import Modules
    - Set Image Size
    - Initialize Camera (RGB888 Format)
    - Initialize Display and Start Camera
    - Set Running Mode
    - Image Processing
    - Frame Rate Statistics and Resource Release
  - Parameter Adjustment Notes

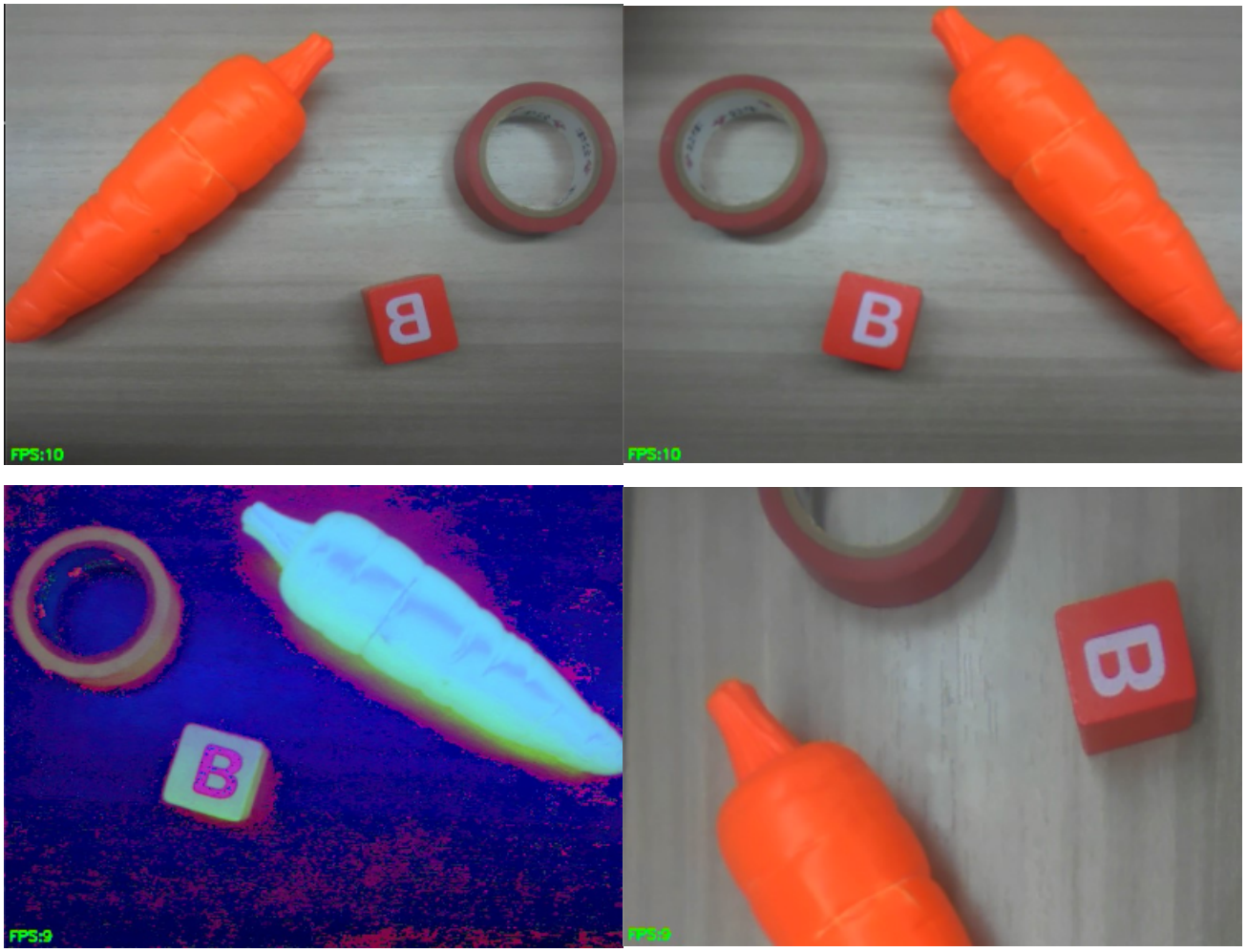
Preface

This tutorial applies to firmware version: CanMV_K230_YAHBOOM_micropython_v1.8-13. Download from the official Canaanie website at https://download.kendryte.com/developer/releases/canmv_k230_micropython/daily_build/

Example Results

Run the example code for this section: 01.camera_cvtColor_resize.py.

This section uses the K230 camera to capture real-time 640×480 RGB888 images, and demonstrates horizontal flip, scaling, affine rotation, and HSV color space conversion by modifying the mode. The processed results are displayed on the ST7701 screen and synced back to CanMV IDE.



Principles

Why Can Images Be Processed by OpenCV

`Sensor.snapshot()` returns a CanMV `image.Image` object, and `to_numpy_ref()` exposes the image buffer to `cv2` as a `ulab.numpy.ndarray` view. It references the same frame buffer instead of creating a copy of the image, making it suitable for real-time processing.

The example uses the following data flow:

```
Camera frame image.Image
         ↓ to_numpy_ref()
RGB888 ndarray
         ↓ cv2 transform
Processed result written back to img_np
         ↓ Display.show_image(img)
Screen and IDE display
```

Four Processing Modes

mode	Operation	Core Function	Processing Result
0	Horizontal flip	flip	Image mirrored left-right
1	Downscale then upscale	resize	First reduce to 320×240, then restore to 640×480
2	Rotation and scale	getRotationMatrix2D, warpAffine	Rotate 45° around center and scale to 0.7
3	BGR to HSV	cvtColor	Get HSV three-channel data

In HSV mode, the colors appearing abnormal on screen is normal because the display still treats H, S, V values as ordinary color channels. HSV data is mainly used for subsequent color segmentation, not for direct viewing.

Code Overview

Import Modules

```
import time, os, sys, gc
from media.sensor import *
from media.display import *
from media.media import *
import cv2
import ulab.numpy as np
```

- `media.sensor`: Configure camera and capture images.
- `media.display`: Drive ST7701 screen and IDE image feedback.
- `cv2`: OpenCV MicroPython binding in CanMV firmware.
- `ulab.numpy`: Lightweight array for data exchange with `cv2`.

Set Image Size

```
W, H = 640, 480
```

OpenCV size parameters are usually in "width, height" order, while array shapes are usually in "height, width, channels" order. Do not confuse them when using.

Initialize Camera (RGB888 Format)

```
sensor = Sensor(width=1280, height=960, fps=90)
sensor.reset()
sensor.set_framesize(width=W, height=H)
sensor.set_pixformat(Sensor.RGB888)
```

The construction parameters select 1280×960@90 sensor input mode, and `set_framesize()` sets the output channel to 640×480. cv2 examples use RGB888 three-channel arrays.

Initialize Display and Start Camera

```
Display.init(Display.ST7701, width=640, height=480, to_ide=True)
sensor.run()
time.sleep(0.5)
```

After starting, wait 0.5 seconds for the sensor and display pipeline to stabilize.

Set Running Mode

```
mode = 2
```

The source code default is 2. After changing the mode, you need to stop and rerun the script. The program will not automatically switch functions.

Image Processing

```
img = sensor.snapshot()
img_np = img.to_numpy_ref()

if mode == 0:
    img_np[:, :, :] = cv2.flip(img_np, 1)
elif mode == 1:
    small = cv2.resize(img_np, (320, 240))
    img_np[:, :, :] = cv2.resize(small, (W, H))
elif mode == 2:
    matrix = cv2.getRotationMatrix2D((W // 2, H // 2), 45, 0.7)
    img_np[:, :, :] = cv2.warpAffine(img_np, matrix, (W, H))
else:
    img_np[:, :, :] = cv2.cvtColor(img_np, cv2.COLOR_BGR2HSV)
```

`img_np[:, :, :] = result` writes the operation result back to the original frame, ensuring that `Display.show_image(img)` displays the processed image, while avoiding holding an additional full-size output array for a long time.

Frame Rate Statistics and Resource Release

```
if time.ticks_diff(time.ticks_ms(), ft) >= 1000:
    print("fps:", fc)
    fc = 0
    ft = time.ticks_ms()

finally:
    sensor.stop()
    Display.deinit()
    os.exitpoint(os.EXITPOINT_ENABLE_SLEEP)
    time.sleep_ms(100)
```

The `finally` block ensures that the camera and display modules are closed when the script is stopped by the IDE or when an exception occurs, facilitating the next run.

Parameter Adjustment Notes

- **mode**: It is recommended to first test 0, 1, 3 to confirm that shared frame write-back, scaling, and color conversion work correctly.
- **Scaling size**: Smaller internal size means faster speed, but more detail loss after upscaling. You can compare 320×240 with 160×120.
- **Rotation angle**: Positive value means counterclockwise rotation; when scale factor is less than 1, the image shrinks.
- **Current firmware limitation**: The project board has confirmed that the `CV_64F` return matrix of `getRotationMatrix2D()` has a type mapping defect in the current firmware, which may produce abnormal values and cause out-of-bounds write risk. Before the firmware is fixed, do not run `mode=2` for a long time; you can change the default mode to 0 first.
- **Memory management**: The example executes `gc.collect()` every 10 frames. If frame capture fails, you can first turn off `to_ide` or reduce the processing resolution for comparison.